

개발자를 위한 이력서 포트폴리오 완벽 가이드 2

개발자의 향로를 더 선명하게 비추는 빛

2026.02

생각등대

생각등대: 개발자를 위한 이력서 포트폴리오 완벽가이드 2

안녕하세요. '생각등대' 입니다. 생각등대: 개발자를 위한 이력서 포트폴리오 완벽가이드 시리즈 2편이 출간되었습니다. 저는 타 플랫폼에서 수많은 멘토링을 진행했고 부트캠프에서 이력서, 포트폴리오, 모의면접 코치로 활동하며 많은 이력서를 검토해 멘티분들의 합격을 도왔습니다.

많은 멘토링을 계속하면서 멘티분들의 가장 공통적으로 힘들어하는 부분이 '이력서와 포트폴리오 관계 그리고 작성 방법' 입니다. 처음 취업을 준비하시는 분, 재직 중 이직을 고려하시는 분, 빠르게 이력서와 포트폴리오를 완성하고 싶은 분들을 위해 준비한 전자책입니다.

시리즈 1편과 큰 틀은 동일합니다. 다만 그 사이 저 역시 더 깊이 연구했고 타 기업에서의 이력서, 포트폴리오 코칭 경험과 수백 명의 멘티 그리고 오픈카카오톡에서 만난 많은 분들의 사례를 통해 더 디테일한 내용을 담아 시리즈 2편을 출간하게 되었습니다.

현재 시중의 어떤 공략집보다도 도움이 될 것이라 자부합니다. 닉네임처럼 여러분의 길을 밝히는 빛이 되어드리겠습니다.

이 전자책을 읽으면 도움이 될 분들입니다.

1. 이력서와 포트폴리오를 써본 적이 없어요.
2. 빠르게 이력서와 포트폴리오를 만들고 싶어요.
3. 이력서와 포트폴리오 방향성은 잡았는데 방향성만 잡았고 디테일을 못잡았어요.
4. 이력서와 포트폴리오를 완성했는데 합격률이 저조해요.
5. 다른 지원자들보다 더 역량있게 쓰고 싶어요.
6. 멘토링을 받고 싶지만 부끄러워요.

이 전자책을 완독하면 이력서, 포트폴리오를 비롯한 취업 준비 전반의 기준이 정리되어, 더 이상 갈팡질팡하지 않고 혼란 없이 깔끔하게 준비할 수 있도록 하는 것이 저의 목표입니다.

다음은 목차입니다.

목차

1장 : 이력서 실전 정보

- 1. 이력서를 검토하는 면접관의 심리
- 2. 이력서는 어떤 틀로 써야할까요?
- 3. 이력서 쓰는 방법 : 제목, 사진, 지원 동기, 자기소개, 프로필
- 4. 이력서 쓰는 방법 : 프로젝트
- 5. 이력서 쓰는 방법 : 자격증, 대외 활동 그 외

2장 : 포트폴리오 실전 정보

- 1. 포트폴리오의 역할. 이력서와 어떤 차이가 있나요?
- 2. 포트폴리오 쓰는 방법 : 제목
- 3. 포트폴리오 쓰는 방법 : 그림, 문제 해결
- 4. 문제 해결 경험이 아닌데, 포트폴리오에 써도 되나요?

3장 : 자기소개 실전 정보

- 1. 자기소개서 쓰는 방법
- 2. 합격률을 높이는 필살기

4장 : 다른 지원자와 차별화되는 '나만의 무기' 만들기

- 1. 뻔한 블로그를 나만의 무기로 만드는 방법
- 2. 이력서의 소개 섹션을 나만의 무기로 만드는 방법

5장 : 지원 전략과 AI 시대 생존법

- 1. 지원하는 방법. 아직도 플랫폼에서 지원하시나요?
- 2. AI 로 인해 어떻게 살아남아야 할까?

자, 이제 본격적으로 들어갑니다.

많은 분들이 떨어지는 이유는 실력이 아니라 이력서, 포트폴리오에 대한 잘못된 편견 때문이었습니다. 지금 가진 기준을 잠시 비우고, 이 책이 제시하는 기준으로 다시 세팅해봅시다.

1장 : 이력서 실전 정복

1-1) 소재목 : 이력서를 검토하는 면접관의 심리



[이미지] 이력서를 보는 면접관의 모습

취준생, 이직 준비생분들께서 가장 먼저 하는 일은 보통 이력서 작성입니다. 여기저기 검색하며 “합격했다”는 사람의 이력서 템플릿을 찾아보고 그 틀에 내가 해온 일들을 채워 넣기 시작하죠. 저 역시 취준할 때도, 첫 이직을 준비할 때도 가장 먼저 한 일이 이력서를 만드는 것이었습니다.

그런데 그 이력서의 각 항목을 왜 써야 하는지, 하나하나 제대로 생각해보신 적 있으신가요? “다들 이렇게 쓰나 보네, 나도 이렇게 써야지.” 그렇게 생각한 채, 이유는 모른 채 내용만 채우느라 바쁘지는 않았나요?

이 챕터에서는 이력서를 쓰기 전에, 이력서를 읽는 면접관의 상황과 심리부터 이해합니다. 그래야 지피지기처럼 더 정확하게 이력서를 설계할 수 있습니다.

요즘 채용 시장은 수요보다 공급이 훨씬 많습니다. 그만큼 면접관은 엄청난 수의 이력서를 빠르게 검토해야 합니다. 실제 면접관 입장에서 보면, 따로 시간을 내어 하루에 10~20개 정도의 이력서를 보는 경우도 흔합니다.

그리고 면접관의 일이 이력서 검토로 끝나지 않습니다. 과제 전형이나 코딩테스트가 있는 회사라면 과제와 코테 결과도 함께 확인해야 하고, 동시에 면접도 진행해야 합니다. 이런 면접관의 상태를 알아야, “읽히는 이력서”를 만들 수 있습니다.

또 한 가지 더 짚고 넘어가야 합니다. 공고는 왜 올라왔을까요? 대부분은 팀 인원 대비 일이 많고 당장 사람이 필요한 상황이기 때문입니다. 이런 경우 이력서를 보는 면접관 역시 일정이 꽉 차 있고 야근을 하고 있을 가능성도 높습니다.

정리하자면, 면접관은 지금 동시에 처리해야 할 일이 너무 많습니다. 그래서 이력서에 쓸 수 있는 시간은 짧고 컨디션도 좋은 편이 아닐 수도 있습니다. 결국 이력서는 빠르게 판단될 수 있도록 읽기 좋은 문서가 되어야 합니다.

취준을 하느라, 이직을 준비하느라 이러한 상황을 고려하지 못한 채 곧바로 이력서와 포트폴리오를 작성하면 방향성이 잘못돼 결과가 좋지 않은데, 그건 당연한 겁니다. 이제는 이러한 상황을 충분히 이해했고 방향성을 잡았으니 이력서를 써보러 가봅시다.

1장 : 이력서 실전 정복

1-2) 소재목 : 이력서는 어떤 툴로 써야할까요?



[그림] 이력서를 만드는 툴

이제 면접관의 심리를 이해했다면 그 전제를 바탕으로 이력서를 작성하면 됩니다. 그런데, 그전에 우리는 이력서의 툴을 만들어야합니다. 이력서 작성에는 다양한 포맷이 있고 툴 역시 여러 가지가 있습니다. 수백 명의 인프런 멘토링과 교육기관 이력서 코치 경험 그리고 면접관 경험을 종합해보면 보통 Google Docs, 워드, 노션, 피그마, 캔바, 플랫폼 이력서 등을 많이 사용합니다.

피그마와 캔바는 원래 디자인 툴이지만, 요즘은 이력서 작성에도 흔히 활용됩니다. 그렇다면 이력서는 어떤 툴로 작성하는 것이 가장 좋을까요?

이러한 경험 데이터를 기반으로 봤을 때, 딱 한가지만 빼면 전부 다 괜찮습니다. 그 한가지는 플랫폼의 이력서를 그대로 쓰는 겁니다. 저희가 흔히 알고 있는 여러 플랫폼이 있죠? 플랫폼마다 정해진 이력서의 툴이 있습니다. 그 툴을 제외한 Google Docs, 워드, 노션, 피그마 그리고 캔바까지 전부 사용해도 무방합니다.

왜, 플랫폼 툴을 지양해야 할까요? 그건 면접관의 경험을 생각하시면 됩니다. 면접관은 하루에 10개 정도 되는 이력서를 봅니다. 그런데 그 이력서들이 대부분 빠르게 지원할 수 있는 플랫폼 이력서 기반인 경우가 많습니다. 그러면 면접관은 딱 이런 느낌이란 똑같아요.

‘겨울에 전부 다 똑같이 룽패딩을 입은 느낌이군. 다 그냥 똑같군.’

이러한 느낌이 들게 하는 순간, 끝입니다. 수요가 많고 공급이 많은 시장에서는 무조건 유니크해야 합니다.

자, 그럼 이 글을 읽고 있는 여러분은 플랫폼 이력서가 아닌, 자신만의 이력서를 앞서 나온 툴을 이용해 만들면 됩니다.

1장 : 이력서 실전 정복

1-3) 소제목 : 이력서 쓰는 방법 : 제목, 사진, 지원 동기, 자기소개, 프로필

생각등대 이력서 (멘토링 교육용)

Introduction

지원 동기

Java 기반의 웹 백엔드 기술을 중심으로 실무형 프로젝트와 교육을 수료한 신입 개발자입니다. JSP, Spring Framework, jQuery를 활용한 웹 기능 개발 경험이 있으며, 실습 과정에서 Git과 Maven을 이용한 협업과 빌드 경험도 보유하고 있습니다. 그 룹웨어처럼 다양한 사용자 요구에 맞춘 기능 커스터마이징과 유지보수가 중요한 서비스에서 실용적이고 유연한 개발자로 성장하 고 싶어 지원하게 되었습니다.

책임감 있는 개발자

팀 프로젝트를 진행하며 일정 관리에 대한 책임감을 가장 중요한 가치로 생각하게 된 개발자입니다. Redis 기반의 캐시 처리와 MySQL 데이터 모델링, Query 튜닝에 강점을 가지고 있으며, 실습 중심의 프로젝트에서 백엔드 구조를 설계하고 구현하는 경협을 쌓아왔습니다. 팀 내에서는 주어진 일정을 끝까지 책임지고 마무리하며 기획자, 디자이너와의 원활한 커뮤니케이션을 통해 팀 원들로부터 신뢰를 받아왔습니다.

Blog Link : 레디스, 나는 끝까지 파고들었다. 레디스, 더 깊이 들어갈 데가 없다.

Personal Information

NAME : 생각등대

Company (or Education)

- B_생각등대 컴퍼니 (취업 개발팀) : 2025.06 ~ 현재
- A_생각등대 컴퍼니 (취업 개발팀) : 2020.12 ~ 2022.10
- 생각등대 부트캠프 (Java 기초 및 실무 교육) : 2020.02 ~ 2020.05

GITHUB : 생각등대 GitHub URL

BLOG : 생각등대 Blog URL

PORTFOLIO : <https://think-lighthouse.com/> (본인이 선택한 포트폴리오 URL)

[그림] 이력서 최상단 - 사진, 지원동기, 자기소개, 프로필

자, 그럼 이제 본격적으로 시작해볼까요? 우선, 이력서의 전체적인 부분이 아닌 최상단 카테고리에 대해서 설명드리려고 합니다.

첫번째 방향성은 ‘제목’ 부분입니다. 해당 카테고리는 간단하게 ‘이름 이력서’라고 쓰면 됩니다. 많은 분들이 제목에 “열정적인 개발자 생각등대입니다.” 또는 “코드에 진심인 개발자 생각등대입니다.”와 같은 문구를 쓰는 경우가 많습니다. 이런 표현은 무조건 지양하길 바랍니다.

개발자 붐이었을 때는 수요가 많아 이런 사소한 부분이 크게 문제 되지 않았습니다. 하지만 면접관 경험상, 이런 식으로 팩트 기반이 아닌 그 외적인 것을 어필하는 지원자에게 선입견을 갖는 면접관도 꽤 있습니다. 즉, 굳이 불필요한 문구를 덧붙이지 않는 것을 추천합니다.

두번째 방향성은 ‘사진’ 부분입니다. 사진은 가능하면 꼭 넣는 것을 추천드립니다. 결국 팀은 ‘일을 같이할 사람’을 뽑는 곳이니깐요.

팀에서는 보통 역량에 대한 기준선이 있습니다. 예를 들어 기준선을 60점이라고 해보겠습니다. 김맑음 지원자와 민슬품 지원자가 후보에 올랐고, 김맑음 지원자는 역량 점수로 70점을 받았습니다. 민슬품 지원자는 80점을 받았어요. 그런데 김맑음 지원자의 이력서 사진을 보니 밝아 보이고, 성실해 보이고, 함께 일하고 싶은 인상이 듭니다. 반면 민슬품 지원자는 사진이 없습니다.

이때 어떤 지원자를 뽑을까요? 저라면 ‘같이 일하고 싶은’ 김맑음 지원자를 뽑겠습니다. 이유가 무엇일까요?

이유는 간단합니다. 역량이 기준선만 넘으면, 그 이후에는 ‘기술 점수’보다 ‘함께 일하고 싶은 사람인가’ 같은 비기술 요소가 더 크게 작용하기 때문입니다. 즉, 기준선을 통과한 순간부터는 태도, 인상, 커뮤니케이션, 협업 이미지 같은 부분이 더 많은 점수를 받습니다.

그럼 어떤 사진을 넣어야 할까요? 지원할 때는 그 회사의 분위기와 문화를 먼저 파악하는 게 중요합니다. 보편적으로 SI 회사는 보수적이고 수직적인 문화인 경우가 많습니다. 이런 경우에는 깔끔하게 나온 증명사진을 넣는 것이 가장 무난합니다.

반면 스타트업이나 서비스 중심 회사는 분위기가 조금 더 자유롭고 사람 간 소통을 중요하게 보는 경우가 많습니다. 이런 곳에는 여행에서 찍은 자연스러운 사진처럼 너무 딱딱하지 않은 사진이 더 잘 맞을 수 있습니다. 스타트업, 서비스 회사는 조직 문화 자체가 비교적 수평적인 경우가 많기 때문입니다.

세번째 방향성은 ‘지원 동기’ 부분입니다. 저는 면접관 경험을 하고 나서 제 이력서에 지원 동기를 반드시 쓰기 시작했습니다. 제가 100명 이상의 지원자를 면접해보며 느낀 점은, 이력서에 ‘지원 동기’를 적는 지원자가 거의 없었다는 것입니다. 그래서 지원 동기를 써 둔 지원자를 보면 유독 눈에 띄고, 더 정성스럽게 보였습니다.

면접관 입장에서는 지원자들이 여러 회사에 동시에 지원한다는 걸 알고 있습니다. 그런데도 지원 동기를 통해 “내가 도움을 줄 수 있어요”를 보여주는 지원자는 확실히 유니크합니다. 그리고 역량이 기준선을 넘는다면, 이런 지원자는 뽑고 싶은 마음이 강하게 듭니다.

자, 그럼 이력서의 지원 동기는 어떻게 써야 할까요? 유튜브나 블로그 글을 보면 보통 “회사를 분석하세요, 서비스를 분석하세요, 매출을 얼마나 올리겠습니다” 같은 예시로 지원 동기를 쓰라고 합니다. 그런데 개발자 직무만큼은 그렇게 쓰지 않는 편이 좋습니다.

“해당 팀에 어떤 도움을 줄 수 있는지 쓰세요.”

그럼 어떻게 쓰면 될까요? 채용 공고의 지원 자격과 우대 사항을 보면 여러 기술과 요구사항이 적혀 있을 겁니다. 그 내용을 읽었을 때, 머릿속에 “내가 들어가면 어떤 구조로, 어떤 일을, 어떻게 해결할지” 아키텍처가 그려져야 합니다. 자, 이미지 예시를 한 번 볼까요?

| 이런 분과 함께 하고 싶어요

- 웹 애플리케이션 개발, 운영 경험자로서 실무 4년 차 이상 또는 그에 준하는 역량을 보유한 분
- Java에 익숙하고 기타 언어 하나 이상 사용이 가능한 분
- SpringBatch를 이용한 batch job 개발 경험이 있는 분
- Spring 프레임워크를 이용한 Web Application, RESTful API 개발 경험이 있는 분
- Microservices 아키텍처 기반의 시스템 개발 경험이 있는 분

| 이런 분이면 더 좋아요

- 정산/회계 서비스 개발 경험이 있는 분
- E-Commerce, O2O 서비스 개발 경험이 있는 분
- 테스트 코드 작성이 습관화되어 있는 분
- Kafka, Kafka Streams를 이용한 실시간 데이터 처리 경험이 있는 분
- AWS를 이용한 개발, 상용 서비스 운영 경험이 있는 분
- JPA, Hibernate 등 ORM 사용과 도메인 모델링 경험이 있는 분
- 웹 프론트엔드 개발 능력과 UI에 대한 기본 지식과 경험이 있는 분
- 새로운 기술 도입에 거부감이 없고, 개발 문화 개선에 적극적인 분

[그림] A 회사의 지원자격과 우대 사항

위의 그림은 A 회사의 채용 공고입니다. 이 공고를 본 지원자는 머릿속에 해당 팀의 아키텍처가 그려져야 합니다. 즉, 어떤 구조에서 어떤 일을 하게 될지 감이 와야 합니다. 그리고 지금까지 내가 했던 프로젝트, 즉 이력서에 적어둔 경험 중에서 무엇이 이 공고와 ‘연관성’이 있는지 확인해야 합니다.

확인 후, 이력서의 지원 동기 카테고리에 ‘내가 이러한 프로젝트를 했으니 당신의 팀에 들어가면 어떠한 기여를 할 수 있습니다.’ 하고 2줄에서 3줄 정도 작성하면 됩니다. 이 지원 동기는 굉장히 중요합니다. 이력서 뿐 아니라 면접에서도 ‘우리 회사에 왜 지원했나요?’ 라는 가장 어려운 질문에 답변을 제대로 할 수 있습니다.

자, 그럼 여러 가지 케이스가 있을 수 있습니다. 만약 해당 공고와 ‘연관성’이 있는 프로젝트가 없다면 어떻게 해야 할까요? 결론부터 말하면, 쓰지 않는 게 좋습니다. 써 봐야 서류나 면접 과정에서 금방 드러나고, 결국 탈락할 가능성이 높습니다.

반대로 다른 케이스도 있습니다. 지원 자격 중 일부 기술은 부족하더라도, 팀에 실질적으로 도움을 줄 수 있는 근거가 있다면 지원 동기에 써도 됩니다. 예를 들어, kafka 경험은 없더라도 공고에 적힌 다른 요구사항 (예: Spring Batch, 정산 도메인 경험, AWS 등)에서 내 경험이 도움이 될 수 있다면, 그 부분을 중심으로 작성하는 겁니다. 이해하셨죠?

네번째 방향성은 ‘자기소개’ 부분입니다. 이 카테고리는 지원자 유형이 3가지로 크게 나뉩니다.

첫번째 유형은 열정과 노력을 강조하는 사람입니다. “무언가를 열심히 해왔다”, “끝까지 최선을 다한다”처럼 태도와 성향을 어필합니다. 두번째 유형은 꼼꼼함과 책임감을 강조하는 사람입니다. “디테일을 놓치지 않는다”, “완성도를 중요하게 본다” 같은 강점을 내세웁니다. 그리고 마지막으로 팩트 기반으로 쓰는 사람도 있습니다. 예를 들어 오픈소스 기여 경험이나, 프로젝트에서 DAU/MAU 같은 수치를 활용해 성과를 구체적으로 보여주는 방식입니다.

어떤 유형으로 써야 할까요? 이건 단언할 수 있습니다.

이제는 1일 1커밋, 1주 1포스팅 같은 것을 어필하는 건 아무 도움이 되지 않습니다. 아니, 오히려 마이너스 편견이 생기기도 합니다. 보여주기식으로 느껴지기 때문이죠. 또한 꼼꼼함, 책임감 같은 부분은 면접에서 직접 대화를 통해 충분히 느낄 수 있습니다. 이력서처럼 글로만 적혀 있는 내용은 신뢰도가 떨어질 수밖에 없습니다. 그래서 면접관 입장에서는 **마지막 유형인 팩트 기반으로 쓰는 방식**이 훨씬 더 신뢰감 있고, 역량도 있어 보입니다. 가능한 한 팩트 기반으로 작성해 주세요.

마지막 방향성은 ‘프로필’ 부분입니다. 간단히 이름, 이메일, 경력, GitHub, 블로그, 포트폴리오 링크를 달아주면 됩니다. 이름과 이메일은 당연히 작성해야 하고요.

여기에 간혹 주소까지 적는 분들이 계십니다. 그런데 주소는 오히려 단점이 될 수 있습니다. 예를 들어 지방에 살지만 합격하면 상경할 계획인 분들도 있죠. 하지만 이력서에 현재 거주지 주소를 적어두면, 면접관은 “합격하면 올라오겠지?”라고 생각하지 않습니다. 즉, 주소는 적지 않는 것을 추천드립니다.

자, 이제 경력 표기에 대해 말씀드리겠습니다. 생각보다 많은 분들이 이 부분을 놓칩니다. 예를 들어 ‘2년 차 채용 공고’가 있다고 해볼게요. 이 공고에는 신입, 1년 차, 2년 차, 3년 차, 4년 차까지도 지원합니다. 면접관 입장에서는 크게 문제되지 않습니다. 기준이 되는 역량이 있고, 그 기준을 넘는 사람을 고르면 되니까요.

다만 면접관이 생각보다 힘들어하는 포인트가 있습니다. 바로 이력서 첫 페이지를 보자마자 “이 지원자가 몇 년 차지? 신입인가?”를 바로 알 수 없는 경우가 많다는 점입니다. 이럴 때 면접관은 스크롤을 내리며 계속 추측해야 합니다. 그래서 첫 페이지의 프로필란에 경력(또는 교육 이력)을 명확하게 적어, 면접관이 첫눈에 바로 파악할 수 있도록 써주세요.

포트폴리오 링크를 다는 이유는, 간혹 제출해야 하는 첨부 파일이 한 개로 제한되는 경우가 있기 때문입니다. 그럴 때는 포트폴리오를 링크로 달아 이력서를 제출해 주세요. 또는 홈페이지에 이미 이력서와 동일한 내용을 충분히 적는 란이 있으면 그때는 포트폴리오만 제출해도 괜찮습니다.

블로그와 GitHub은 사실 깊게 들여다볼 시간이 많지 않습니다. 그래서 간단하게만 관리해주시면 됩니다. 블로그는 1주 1포스팅까지는 필요 없고, 한 달에 1포스팅 정도만 해도 충분합니다. GitHub은 가능한 한 공개로 열어두시고, 코드 디테일을 과하게 신경 쓰기보다는 큰 틀에 집중하는 게 좋습니다. 예를 들어 패키지 구조는 실무에서 자주 쓰는 명확한 형태가 있습니다. 그런 구조를 참고해서 개발하시고, 처음 보는 이상한 패키지 구조로만 짜지 않으면 됩니다.

1장 : 이력서 실전 정복

1-4) 소제목 : 이력서 쓰는 방법 : 프로젝트

Project

[생각등대 회사 또는 교육] 프로젝트 제목

← 기본 틀

일정 : 2024.05 ~ 2024.11

기술 스택 : 사용했던 스킬 나열

참여 인력 : 해당 포지션에 프로젝트를 완수하기 위한 참여 인력

서비스 : 프로젝트에 대한 개요

- 문제 + 해결 + 결과 + 도메인
- 문제 + 해결 + 결과 + 도메인
- 문제 + 해결 + 결과 + 도메인
- 문제 + 해결 + 결과 + 도메인
- 단순히 구현
- 단순히 구현
- AWS 인프라적인 부분 구축 경험
- 팀 내에서 개발적인 부분을 제외한 역량

[그림] 이력서의 꽃 '프로젝트' 카테고리 - 템플릿

이력서의 꽃인 프로젝트에 대해 설명드리겠습니다. 위 그림은 '생각등대 컴퍼니 이력서 템플릿'의 프로젝트 카테고리 기본 틀입니다. 제가 지향하는 방향은 신입, 주니어 분들은 '프로젝트' 단위로 이력서를 작성하는 것이고, 5년 이상 경력이 있는 분들은 회사 단위 또는 큰 프로젝트 단위로 정리해 나열하는 것이 가장 좋다고 생각합니다.

첫 번째 방향성은 프로젝트 개수입니다. “멘토님, 프로젝트는 몇 개 정도 써야 할까요?”라는 질문을 정말 많이 받습니다. 사실 이 질문에는 정답이 없습니다.

그래서 저는 면접관 경험을 바탕으로, 면접관 입장에서 어떤 이력서를 선호할까를 기준으로 말씀드립니다. 결론부터 말하면, 프로젝트는 3개, 많아도 4개 정도면 충분하다고 생각합니다. 위 그림의 예시가 1개의 프로젝트라고 했을 때 기준으로 3~4개를 작성하면 이력서는 보통 2장 내외가 나오는데, 이 정도가 면접관이 읽을 때 가장 무난하고 부담이 적은 분량입니다.

가끔 프로젝트를 2개만 작성하시는 분들도 있는데, 멘토링을 해보면 그 2개만으로도 충분한 지원자들이 분명 있습니다. 다만 그 경우는, 두 프로젝트의 완성도와 임팩트가 확실히 높은 경우에 가능합니다. 만약 그렇지 않다면, 2개에서 부족하거나 아쉬운 부분을 보완해 줄 세 번째 프로젝트를 추가하는 것이 더 좋은 방향입니다.

두 번째 방향성은 제목, 일정, 기술 스택, 참여 인력, 서비스 카테고리입니다. 먼저 제목부터 말씀드리면, 지금까지 진행했던 프로젝트에 제목을 붙여주시면 됩니다. 이때 가장 좋은 방식은 제목만 봐도 어떤 프로젝트인지 바로 이해되도록 작성하는 것입니다.

예를 들어 프로젝트 제목을 단순히 “Thinklighthouse”라고 지으면, 제목만으로는 어떤 프로젝트인지 감이 잘 오지 않는 경우가 많습니다. 반면에 “취업, 이직, 1인 브랜딩을 위한 교육 프로젝트”처럼 쓰면 프로젝트의 성격과 목적이 명확하게 전달됩니다. 이런 방식으로 작성하시는 것을 추천드립니다.

그럼 현재 재직 중인 상태에서 이직 준비를 할 때는 어떻게 써야 할까요? 프로젝트 제목은 회사에서 정해주는 것이 아니라, 본인이 이해하기 쉽도록 임의로 만들어 작성하는 것입니다. 즉, 1년 동안 하나의 서비스만 운영하거나 담당했다면, 저는 공통된 업무를 묶어서 하나의 프로젝트로 정리할 것 같습니다. 예를 들어 “유지보수 프로젝트”, “성능 개선 프로젝트”처럼 말이죠.

일정은 간단합니다. 일 단위로 쓰지 마시고 월 단위로 작성해 주세요. 그리고 가능하면 각 프로젝트는 3개월 내외의 기간으로 정리하는 것을 지향합니다. 사람 심리상 1~2개월로 보이면 프로젝트가 너무 짧게 느껴질 수 있습니다.

기술 스택은 생각보다 많은 분들이 과하게 적는 경우가 있습니다. 불필요한 내용이 포함되는 경우가 많습니다. 예를 들어 Git, Slack, Notion 같은 협업 도구는 굳이 쓰지 않는 것을 추천드립니다. 프로젝트의 핵심 기술을 파악하는 데 큰 도움이 되지 않기 때문입니다. 또 한 가지 강조드리고 싶은 부분이 있습니다. 가능하면 기술 스택에 버전을 함께 적어주세요. 예를 들어 Java만 적기보다 Java 17처럼 버전을 명시하면, 면접관 입장에서 사용 기술을 더 빠르고 정확하게 파악할 수 있습니다.

참여 인력은 “같은 포지션”의 인원과 얼마나 협업했는지가 핵심입니다. 사실 팀 프로젝트라고 해도, 해당 포지션이 본인 혼자였다면 실질적으로는 개인 프로젝트와 크게 다르지 않습니다. 예를 들어 백엔드라면, 백엔드 서버를 함께 설계하고 코드를 같이 관리하는 과정에서 충돌이 나거나 의견이 맞지 않을 때 어떤 방식으로 커뮤니케이션하고 조율했는지를 면접관은 보고 싶어합니다.

따라서 참여 인력을 적을 때 여러 포지션을 합쳐서 “10인”처럼 작성하기보다는, “FE 2명 / BE 3명” 처럼 포지션별로 상세하게 적는 것을 추천드립니다.

마지막으로 서비스 개요 부분입니다. 이 항목은 프로젝트에 대한 간단한 설명을 적는 란인데, 사실 크게 중요할까요? 면접관 입장에서 생각해보면, 하루에 이력서를 10개 정도 검토해야 하고, 각 이력서에 프로젝트가 3개씩만 있어도 하루에 서로 다른 프로젝트를 30개나 보게 됩니다. 그래서 면접관은 프로젝트 설명을 길게 읽기보다는 “어떤 도메인에서 어떤 문제를 어떻게 해결했고, 그 결과가 어떻게 나왔는가?”를 가장 중요하게 봅니다. 즉, 프로젝트에 대한 상세한 소개는 과하게 길 필요가 없고, 프로젝트를 한눈에 이해할 수 있도록 간단하게 1줄 정도로만 적어주시면 충분합니다.

[생각등대 부트캠프] 교육 커뮤니티 서비스 개발 ← 예시

일정 : 2025.02 ~ 2024.05

기술 스택 : Spring Boot 3.2, MySQL 8.0, Java17, JPA, MyBatis, JSP, Thymeleaf, Redis, AWS

참여 인원 : 3인

서비스 : 취업하는 모든 분들을 위한 교육 커뮤니티 개발

- 문제 해결 결과 도메인
- 질문 게시글 리스트 조회 시, 느린 응답 문제 발생 및 Redis 캐시 적용으로 평균 응답속도 350ms → 40ms 개선
- 생각등대 게시물 리뷰 댓글 등록 시 트랜잭션 처리 누락으로 데이터 정합성 문제 및 @Transactional 적용 및 계층 분리로 안정화
- 질문 인기 게시글 정렬 쿼리에서 DB Index 튜닝과 정렬 로직 개선 및 2s → 150ms 개선
- OAuth2 기반의 애플 로그인 기능 구현 및 사용자 경험 증대
- 게시글 신고 시, SMTP를 이용한 관리자에게 메일 자동 발송 기능 구현
- Docker 기반 TestContainer 를 활용하여 데이터베이스와 외부 서비스 의존성 테스트 환경 구축
- AWS EC2, S3 환경 운영 배포 파이프라인 구축 및 서비스 배포 안정화
- 기능 명세 및 기획 내용을 문서화 및 커뮤니케이션을 주도하여 협업에 기여

[그림] 이력서의 꽃 '프로젝트' 카테고리 - 실전 예시

마지막 방향성은 이 프로젝트에서 내가 기여한 부분입니다. 앞에서 9페이지에 있던 '기본 틀' 그림이 템플릿이었다면, 이번 그림은 가장 쉬운 예시로 만든 실전편이라고 생각하시면 됩니다. (해당 예시는 이해를 돕기 위한 참고용 예시이니, 그대로 따라 하기보다는 방향만 참고해 주세요.)

프로젝트 내용을 작성할 때, 간혹 '내가 기여한 부분'이라는 주제를 따로 잡고 쓰시는 분들이 있습니다. 정답은 아니지만, 사실 이력서는 애초에 본인이 한 일을 정리하는 문서이기 때문에 굳이 그 문장을 별도로 명시할 필요는 없습니다. 그런 한 줄이 들어가 있다면 과감히 지워주셔도 됩니다.

자, 한 줄 한 줄을 작성할 때 프로젝트의 최상단에는 "문제 + 해결 + 결과 + 도메인"을 담아 쓰는 겁니다. 면접관은 이력서를 위에서부터 아래로 읽기 때문에, 가장 잘한 것부터 위에 배치하는 것이 중요합니다. 그렇다면 당연히 문제를 어떻게 해결했는지부터 차례대로 나열해야겠죠. 이런 문제 해결 역량을 최소 3줄에서 4줄 정도로 채워주는 것을 지향하시면 좋습니다.

"문제 + 해결 + 결과 + 도메인" 이 키워드는 굉장히 중요합니다. 앞서 언급한 내용이지만, 면접관이 이 한 줄을 읽었을 때 이런 생각이 무의식적으로 떠올라야 합니다. "어떤 도메인(기능)에서 어떤 문제를 어떻게 해결했고, 그 결과가 어떻게 나왔는가?" 이 느낌이 바로 박혀야 한다는 거죠. 도메인이라고 함은 '어떠한 기능' 이라고 생각하시면 이해하기 쉽습니다.

예를 들어 위 예시의 첫 번째 줄인 "질문 게시글 리스트 조회 시 느린 응답 발생 및 Redis 캐시 적용으로 평균 응답 속도 350ms -> 40ms 개선"을 한 번 볼까요? 면접관이 이 문장을 읽었을 때, "교육 커뮤니티의 질문 게시글 조회는 느려질 수 있겠네. 그걸 Redis 캐시로 해결했구나. 40ms 정도면 충분히 타당한 수치네."라고 자연스럽게 납득할 수 있어야 합니다.

즉, 읽는 순간 이해되고, 해결 방식과 결과가 설득력 있게 느껴지도록 작성해야 합니다. 이렇게 하나의 프로젝트에 문제 해결이 3개 정도 있는 것이 가장 좋은 방향성이고 그걸 최상단에 써주시면 됩니다.

이제 문제 해결 내용을 3줄 정도 작성했다면, 그다음부터는 이 프로젝트에서 단순히 구현한 기능도 작성해도 됩니다. 다만 주의하셔야 할 점은 “단순히 구현했음”처럼 끝내면 안 된다는 것입니다.

예를 들어 “애플 소셜 로그인 구현”이라고만 쓰고 끝내면 임팩트가 없습니다. 위 예시의 4번째 줄을 보면 “OAuth2 기반의 애플 로그인 구현 및 사용자 경험 증대”라고 되어 있죠? 이런 식으로, 단순 구현이라도 “어떤 기술, 방식을 이용해서 무엇을 구현했고, 그 결과 어떤 도움이 있었는지”까지 함께 써주셔야 합니다. 이러한 문제 해결이 아닌 ‘구현’은 2줄에서 3줄 정도 써주시면 충분하다고 생각합니다.

마지막에 두 줄 정도는 무엇으로 채워야 할까요? 포지션마다 다르겠지만, 백엔드 기준으로 말씀드리면 본인이 맞닿아 있었던 영역의 역량을 쓰시면 됩니다.

예를 들어 백엔드 직무로 채용 공고에 지원할 때, 프론트엔드와 밀접하게 붙어 있는 전시 파트의 백엔드가 있고, 인프라 팀과 가까운 플랫폼 파트의 백엔드가 있습니다. 이런 경우 전자라면 프론트엔드의 React 라이브러리 또는 Vue 프레임워크와 같은 역량을 적어주면 경쟁력이 될 수 있습니다. 반대로 후자라면 k8s, 배포 자동화, 모니터링, 장애 대응처럼 시스템 기반 역량을 적는 것이 경쟁력으로 이어질 수 있습니다.

이제까지 프로젝트의 개수와 각 카테고리에 대해 설명드렸습니다. 여기까지 읽으신 분들은 프로젝트 카테고리를 어떻게 작성해야 할지 감이 오실 거예요. 위에 있는 예시와 추가로 진행한 템플릿 이벤트를 통해 받은 템플릿 예시를 참고해서 이력서를 작성해 나가시면 됩니다.

마지막으로 가장 중요한 이야기를 할 차례입니다. 이력서를 쓸 때 어느 정도 수준의 역량까지 적어야 할지 감이 안 와서 답답하신 분들이 많을 거라고 생각합니다.

“아... 이걸 되게 쉬운 건데 써야 하나?”

“다른 사람 이력서를 보니까 이걸 너무 쉬운 것 같은데...”

이런 고민을 하는 경우가 정말 많습니다. 정답을 말씀드리겠습니다. 취준생, 주니어라면 연차에 맞는 ‘기본기’가 있습니다. 그 기본기를 60% 정도 채워서 써주세요. 그리고 나머지 40%는 본인의 강점으로 채워주시면 됩니다.

백엔드 취준생 기준으로 예시를 들면, 아래 같은 내용들이 기본기에 해당합니다.

- MySQL Workbench에서 EXPLAIN을 통해 실행 계획을 분석하고, 쿼리를 개선한 경험
- 로컬 캐시를 적용해 API 조회 기능의 응답 속도를 개선한 경험
- Redis 캐시를 활용해 성능을 개선하고 데이터 정합성을 보완한 경험
- 동기/비동기를 구분하고, 비동기 처리를 통해 성능을 개선한 경험

이런 내용들로 60% 정도를 기본기로 채워주시면 됩니다.

그럼, 나머지 40%는 어떤 것으로 채워야할까요?

- JVM 메모리/GC 이슈 분석 힙 영역 비율 및 GC 파라미터를 조정 및 안정성 개선
- tcpdump로 패킷을 분석해 네트워크 장애 원인을 추적 및 대응/재발 방지 조치
- kafka 기반 이벤트 아키텍처를 설계 및 토픽/컨슈머 구조와 처리 전략을 수립
- Netty 기반 SSE를 도입해 실시간 이벤트 전송 구조를 구성 및 안정성 확보

와 같은 내용으로 작성해 주시면 됩니다. 어느 정도 연차가 있으신 분들은 이런 내용들이 어떻게 보면 기본기가 될 수도 있겠죠?

근데 여기서 의문이 들 수도 있습니다. “기본기를 왜 써야 하지?”라고 생각하실 수도 있죠.

면접관 입장에서 보면, 생각보다 많은 신입 분들이 세련되고 화려한 기술이나, 어렵고 복잡한 문제 해결에 대해서는 잘 이야기하는 편입니다. 그런데 기본기에 대해 질문하면, 정말 기본적인 내용인데도 불구하고 모르시는 분들이 꽤 많습니다. 면접관 입장에서는 이런 경험이 이미 많이 쌓여 있을 거예요.

즉, 이력서를 볼 때 “당연히 기본기를 잘 알겠지”가 아니라, 오히려 “기본기는 모르는 거 아니야?”라는 인식이 먼저 깔려 있습니다. 그래서 그 의심을 이력서에서 먼저 해소해 주셔야 합니다.

1장 : 이력서 실전 정복

1-5) 소재목 : 자격증, 대외 활동 그 외

Certifications & Activities

생각등대 컴퍼니 해커톤 (최우수상)

기간 : 2025.06

기술 스택 : Spring Boot, MySQL, Redis, Docker, AWS

- 제한된 시간 내 문제 정의 → MVP 범위 설정 → 핵심 기능 구현까지 빠르게 완주
- API 설계 및 데이터 모델링을 담당하고, 기능 우선순위를 기준으로 개발 순서를 조율
- 트래픽/성능 이슈를 가짐에 캐싱 및 인덱스 적용 등 개선 포인트를 반영하여 안정성을 높임

생각등대 컴퍼니 무료 멘토링 프로그램 '등대항해단'

기간 : 2025.09 ~ 2025.10

- 한 달 단위 기수제로 운영하며, 참여자 모집 → 진행 → 과제 피드백 → 결과 회고까지 전 과정 설계
- 포트폴리오 구성(프로젝트 스토리, 기술 선택 근거, 트러블슈팅 서술 방식)을 표준화하여 템플릿으로 제공
- 참여자 결과물을 기반으로 개선 포인트를 정리해 다음 기수 운영에 반영

SQLD (SQL 개발자)

발급기관 : 한국데이터산업진흥원

취득일 : 2020.08

[그림] 이력서 최하단 - 수상, 대외활동, 자격증

멘토링을 해보면 아직까지도 자격증 준비에 많은 시간을 쓰는 분들이 많습니다. **자격증이 중요할까요?** 이견 경우에 따라 다릅니다. 예를 들어 스타트업이나 서비스 기업만을 목표로 한다면, 굳이 딸 필요가 없습니다. 이렇게 말씀드리면 어떤 분들은 “다들 따니까 안 따면 손해다. 모두가 따는 거면 무조건 따야 한다”라고 이야기하기도 합니다. 하지만 그 의견은 틀렸다고 단언할 수 있습니다. 자격증은 변별력이 없어서, 있다고 해서 평가에서 점수를 더 주는 것도 아닙니다. 즉, 필요 없습니다. 물론, SI는 정보처리기사 필수입니다.

그럼 어떤 경우에 필요할까요? 보편적으로 최신 기술 트렌드에 둔한 회사일수록 자격증을 더 중요하게 보는 경향이 있습니다. 특히 SI나 금융권 회사는 자격증을 우대하는 경우가 많습니다.

대외활동과 수상도 마찬가지입니다. 면접관 경험이 있으신 분들은 어느 정도 공감하실 겁니다. 생각보다 많은 지원자들이 대외활동을 여러 개 했고, 수상 이력도 많습니다. 그런데 내용을 보면 대상, 최우수상 같은 상이 흔하게 등장합니다. 즉, 대외활동과 수상 경력이 많다고 해서, 그 자체만으로는 변별력이 크지 않은 경우가 많습니다. 실제로 멘토링을 하면서 “인상이 깊다”라고 느꼈던 대외활동 수상은 장관상 정도를 제외하면, 대부분은 크게 눈에 띄지 않았습니다. 그래서 대외활동은 ‘얼마나 많이 했는지’보다, 한두 개를 얼마나 ‘깊이 있게’ 했는지가 더 중요해 보입니다.

2장 : 포트폴리오 실전 정복

2-1) 소제목 : 포트폴리오의 역할. 이력서와 어떤 차이가 있나요?

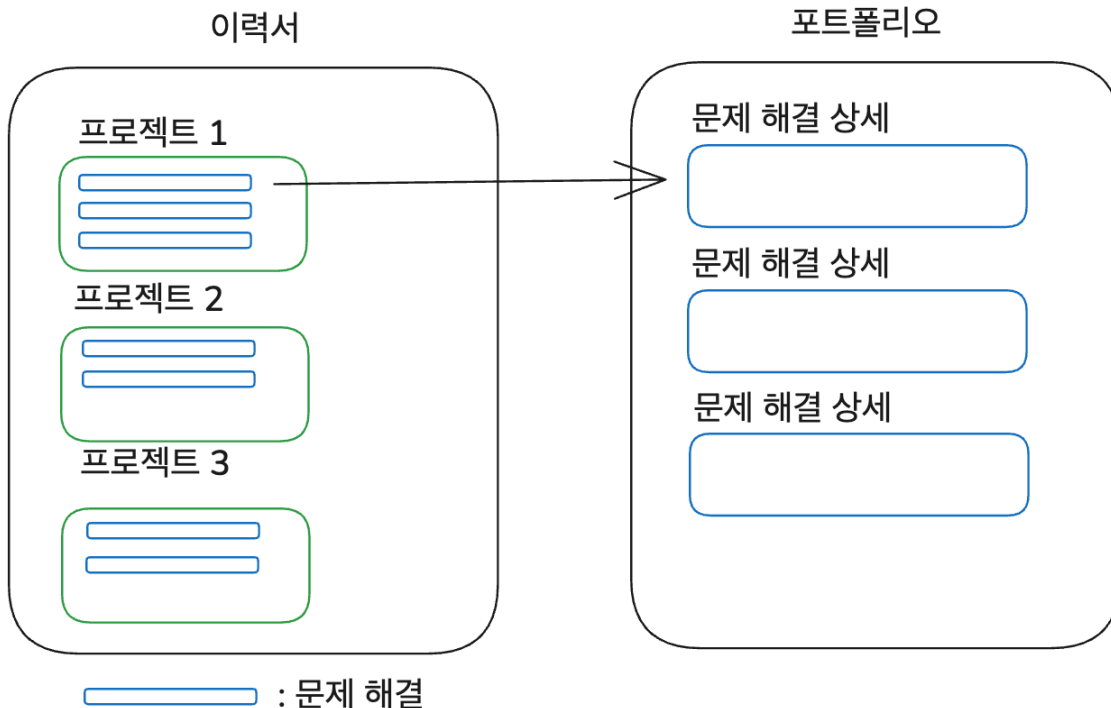
포트폴리오가 대체 무엇일까요? 생각보다 많은 분들이 포트폴리오를 가장 어려워합니다. “아니... 이력서에도 쓰고 포트폴리오에도 쓰면, 이게 이력서랑 무슨 차이야?” 이번 챕터에서는 이력서와 포트폴리오의 차이를 아주 명확하게 설명해 드리겠습니다.

이력서를 다시 한번 복습하고 넘어갑시다. 이력서는 나에 대한 정보를 정리한 문서입니다. 개인정보 부터 내가 어떤 사람인지, 어떤 경력을 쌓았는지, 어떤 교육을 받았는지, 어떤 프로젝트를 했는지까지 전체를 한 번에 정리합니다. 즉, 내가 해온 모든 것이 들어가야 합니다.

여기서 “멘토님, 이걸 다 적으면 이력서가 5장, 많으면 7~8장이 되는 거 아니가요?”라고 생각하실 수 있습니다. 하지만 ‘모든 것을 써야 한다’는 말은 문장을 전부 길게 늘어 놓으라는 뜻이 아닙니다. 최대한 요약해서 핵심만 남기되, 빠짐없이 담아야 한다는 의미입니다.

그래서 제가 앞서 “1장: 이력서 실전 정복”에서 각 프로젝트를 최대한 압축해서 “문제 + 해결 + 결과 + 도메인” 형태로 써 달라고 말씀드린 겁니다. 템플릿과 예시 이미지를 보시면 대부분이 한 줄로 정리되어 있는 것을 확인하실 수 있습니다.

자, 그럼 대체 이력서와 다른 포트폴리오는 무엇일까요? 정답부터 말씀드리겠습니다. **포트폴리오는 이력서에 한 줄로 압축해 둔 각 프로젝트의 “문제 + 해결 + 결과 + 도메인” 내용을 더 자세히 풀어 정리해 둔 문서입니다.** 지금은 아직 감이 잘 안 오실 수 있습니다. 바로 아래에 있는 그림을 함께 보면서 설명드리겠습니다.



[그림] 이력서와 포트폴리오의 관계

위의 그림을 보면서 함께 이해해 보겠습니다. “1장: 이력서 실전 정복”을 보셨다는 전제로 설명드리겠습니다.

이력서에는 각 프로젝트마다, 보통 최상단에 ‘문제 + 해결 + 결과 + 도메인’ 형식으로 3개 정도가 정리되어 있을 겁니다. 이력서에는 한 줄로만 압축되어 들어가 있죠. 그런데 바로 그 한 줄 한 줄이 포트폴리오로 확장할 수 있는 ‘후보’가 됩니다.

예를 들어 프로젝트가 3개라면, 각 프로젝트에 문제 해결 역량이 3개씩 있으니 총 9개가 후보가 되는 셈입니다. 다만 이 9개를 전부 포트폴리오에 쓰는 건 아닙니다. 그 중에서 내가 가장 자신 있고 역량을 확실히 보여줄 수 있는 것만 4개 정도 골라서 포트폴리오로 정리하면 됩니다.

여기서 “뭘 골라야 할지 모르겠다”라고 어려워하시는 분들도 많습니다. 제가 고를 때 사용하는 기준은 간단합니다. 그 문제 해결 과정을 머릿속에 그림처럼 떠올릴 수 있는가, 즉 처음부터 끝까지 흐름이 명확하게 그려지는가를 기준으로 선택합니다.

자, 이제 이력서와 포트폴리오의 차이를 이해하셨나요? 어느 정도 감은 잡으셨을 거라고 생각합니다. 멘토링을 하다 보면 많은 분들이 포트폴리오를 ‘가로형’으로, 마치 PPT처럼 만드시는 경우가 많습니다. 그리고 각 프로젝트 소개와 앱 화면 캡처본을 그대로 붙인 뒤, 그 캡처본에 화살표를 그려 “어떤 기능을 구현했다” 정도만 설명하고 마무리하곤 합니다.

하지만 꼭 말씀드리고 싶은 건, 그런 형태는 포트폴리오가 아니라고 말씀드리고 싶습니다.

2장 : 포트폴리오 실전 정복

2-2) 소제목 : 포트폴리오 쓰는 방법 : 제목

Project Portfolio

예시) 이력서 → 프로젝트 → '문제 해결 역량' 그 한 줄

- 전체적인 아키텍처



- 문제 원인

- 1) 문제 원인 및 문제점 결과
- 2) 문제 원인 및 문제점 결과
- 3) 문제 원인 및 문제점 결과

- 해결 과정

- 1) 해결 과정
- 2) 해결 과정
- 3) 해결 과정
- 4) 해결 과정

- 결과

- 1) 그 결과
- 2) 그 결과
- 3) 그 결과

[그림] 포트폴리오 템플릿

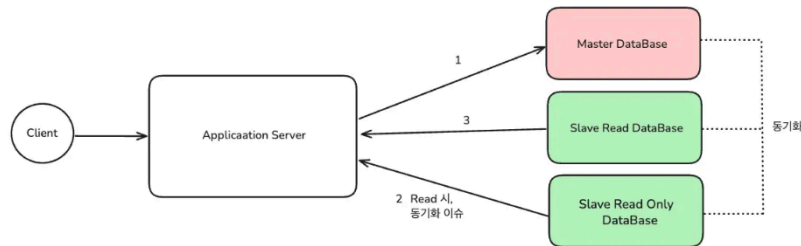
이제 포트폴리오를 본격적으로 작성해볼까요? 위의 그림을 보면서 전체적인 틀을 함께 잡아봅시다.

이전 챕터 '2-1) 소제목: 포트폴리오의 역할. 이력서와 어떤 차이가 있나요?'에서 우리는 이력서와 포트폴리오의 차이점을 살펴봤습니다. 다시 한번 복습해보면, 이력서는 프로젝트 단위로 내용을 짚 나열하는 문서입니다. 예를 들어 프로젝트마다 문제 해결 역량을 3줄씩 작성했고, 프로젝트가 3개라면 문제 해결과 관련된 문장은 총 9개가 나오겠죠. 이 9개 중에서 포트폴리오에 담을 만한 핵심 요소 4~5개를 골라 정리하면 됩니다. 만약 아직 이해가 잘 되지 않는다면, 이전 챕터를 한 번 더 읽어보시는 것을 추천드립니다.

여기서 추출한 이력서의 4~5줄이, 포트폴리오를 쓸 때는 그대로 '제목'이 됩니다. 예시로 한 번 들어보겠습니다.

1. 생각등대 예약 데이터 동기화를 위해 스레드 로컬 활용하여 멀티 데이터소스 간 읽기/쓰기 데이터 일관성 이슈 해결

• 전체적인 아키텍처



• 문제 원인

- 데이터 조회하는 성격에 따라 API 를 분기 처리
- 대량 데이터 처리를 위한 master db (write), 거래 마감 배치용 read-only db, 실시간 조회를 위한 read db 로 역할에 맞게 db 를 분리하여 운영
- 예약과 동시에 데이터를 조회하는 비즈니스 로직에서 빈번한 데이터 일관성 이슈 문제로 알람이 올 때 마다 수동 처리 (약, 하루에 20번 알람, ms 내 에 발생)

• 해결 과정

- 해당 비즈니스 로직만 따로 성능이 다소 낮은 read-only 용이 아닌 read 용으로 data source 변경. 그 외 read-only 그대로 사용
- 커스텀 어노테이션 @Reservation 을 이용하여 해당 비즈니스 로직 API 호출 시, 스레드 로컬에 해당 read 용 구분 명시
- 비즈니스 로직에서 데이터 조회 시, DataSource에서 Connection을 요청할 때 스레드 로컬에 저장된 구분 정보를 기반으로 적절한 데이터 소스를 동적으로 선택하여 연결된 DB에 맞는 조회를 처리하도록 구현
- 테스트
 - 로컬 환경에서 aop + 스레드 로컬을 이용한 api 호출 시, datasource map 으로 관리 후, 로그로 확인
 - docker-compose.yml 를 이용하여 하나의 컨테이너를 임시로 띄운 후, 각각의 API 를 수동으로 테스트 확인

• 결과

- 스레드 로컬을 통한 데이터 소스 동적 변경으로 예약 시 발생하던 데이터 일관성 이슈가 해결되어 알람 발생 빈도 감소 및 시스템 안정성을 크게 향상
- 커스텀 어노테이션과 데이터 소스 동적 변경 방식을 도입. 다른 API들에도 동일한 방식으로 확장 가능하게 되어 시스템 전체의 유연성 높임
- 다양한 비즈니스 로직에 맞게 효율적인 데이터 조회가 가능하도록 개선

[그림] 포트폴리오 예시

위의 예시를 보면 ‘**생각등대 예약 데이터 동기화를 위해 스레드 로컬을 활용해 멀티 데이터소스 간 읽기/쓰기 데이터 일관성 이슈를 해결**’이라는 제목이 있죠. 이 제목은 원래 이력서에서 문제 해결 역량을 한 줄로 정리해 둔 문장이었습니다. 이해가 되었을 거라고 생각합니다. 제목은 이렇게 간단하게만 작성하시면 충분합니다.

2장 : 포트폴리오 실전 정복

2-3) 소제목 : 포트폴리오 쓰는 방법 : 그림, 문제 해결

이전 챕터의 포트폴리오의 제목은 사실상 굉장히 쉬운 파트입니다. 이제부터는 본격적으로 그림과 문제 해결 내용을 어떻게 써야 하는지 알려드릴 차례입니다. 가장 중요한 부분이니 꼭 집중해주세요.

첫 번째 방향성은 ‘그림’입니다. 위의 그림 예시를 보시면, 제목 다음에 ‘전체적인 아키텍처’라는 문구가 들어가 있습니다. 이는 정답이라기보다 예시이므로, 상황에 맞게 유동적으로 바꿔주시면 됩니다.

그런데 이렇게 그림을 넣는 카테고리에 많은 분들이 서비스의 앱 기능을 캡처해서 포트폴리오에 넣는 경우가 정말 많습니다. 백엔드 개발자라면 절대 그렇게 하시면 안 됩니다. 물론 클라이언트 개발자, 퍼블리셔, 프론트엔드 개발자라면 앱 화면 캡처 자체가 강점이 될 수 있어서 충분히 활용 가능합니다. 하지만 백엔드 개발자는 ‘기능 화면’이 아니라 ‘구조와 흐름’을 보여줘야 합니다. 예를 들어 시퀀스 다이어그램, 데이터 흐름도, 문제를 해결한 구간의 아키텍처(변경 전/후) 등을 넣으면 좋습니다. 또한 간혹 테스트 결과를 근거로 제시하는 분들도 있는데, 이것도 아주 좋은 방향이니 적극 추천합니다.

그림을 그릴 때 추천드리고 싶은 툴은 두 가지가 있습니다. draw.io와 excalidraw.com입니다. 이 두 가지 툴만으로도 충분히 작업이 가능하니 적극 활용해보세요. 그리고 그림을 그릴 때 아이콘을 넣는 경우가 있는데, 예를 들어 Docker라면 고래 아이콘을, Spring이라면 나뭇잎 이미지를 넣는 식입니다. 이런 방식은 생각보다 가독성이 많이 떨어집니다.

오히려 말씀드린 2가지 툴을 이용하여 사각형 위주로 구성하고, 각 요소에 Application, Docker처럼 텍스트로 명확히 표기하는 편이 훨씬 전문적이고 좋은 방향성입니다.

그림을 그릴 때 생각보다 어려워하시는 분들이 많습니다. 그런데 제가 보기에는 정답이 따로 있는 건 아닙니다. 데이터 흐름 관점에서 그려도 되고, 요청부터 응답까지의 전체 과정을 한 번에 그려도 됩니다. 화살표 방향이 이쪽인지 저쪽인지 같은 디테일에 너무 오래 고민하느라 시간을 낭비할 필요는 없어요. 본인이 이해하기 쉽고, 전달이 잘 된다고 생각하는 방식으로 그리면 됩니다.

그림 파트에서 마지막으로 당부드리고 싶은 내용은 하나입니다. 모든 문제 해결 항목에 반드시 그림을 포함해 주세요. 즉, 위의 예시처럼 ‘제목, 그림, 문제, 해결, 결과’라는 하나의 덩어리를 정했다면, 그 툴을 끝까지 그대로 유지하셔야 합니다.

만약 다른 문제 해결 사례를 포트폴리오에 쓰려고 하는데, 그 내용을 그림으로 표현할 수 없다면 그 사례는 애초에 포트폴리오에 넣을 만큼의 난이도가 아닐 가능성이 큼니다. 그런 경우에는 다른 사례를 선택하는 편이 좋습니다.

두 번째 방향성은 ‘문제 해결’입니다. 이력서에서 “문제 + 해결 + 결과 + 도메인”을 압축해서 한 줄로 작성했다면, 포트폴리오에서는 그 내용을 더 상세하게 풀어 쓰면 됩니다. 보편적으로는 문제, 해결, 결과를 각각 3줄 정도로 정리해서, 총 9줄 내외로 작성하면 충분합니다. 이 단계에서는 필요하다면 애플리케이션 레벨의 구현 내용까지 포함해도 좋습니다.

간혹 포트폴리오가 아닌 이력서에 애플리케이션 구현 디테일까지 넣는 분들이 있는데, 예를 들어 ApplicationEventPublisher 같은 키워드를 적거나, 낙관적 락을 위해 @Version을 사용한 내용 등을 이력서에 적는 경우가 있습니다. 이런 내용은 이력서보다는 포트폴리오에서 구체적으로 설명하는 편을 추천드립니다.

그런데 막상 이 부분을 쓰기 시작하면, 생각보다 어려워하시는 분들이 많습니다. 이유는 간단합니다. 이력서에는 한 줄로 압축해 적어놨는데, “이걸 어떻게 3줄씩 늘려서 총 9줄로 풀어 쓰냐”에서 막히는 거죠. 그래서 제가 쓰는 팁을 하나 공유드리겠습니다.

저는 제목, 구현한 코드, 기술 스택을 준비한 뒤 LLM에게 이렇게 요청합니다.
“이 데이터를 바탕으로 문제, 해결, 결과를 각각 3줄씩 작성해서 문제 해결 흐름을 정리해줘.
중간에 애플리케이션 레벨의 내용까지 포함해도 좋아.”

이렇게 시켜서 초안을 만들면 생각보다 퀄리티가 좋습니다. 물론 100% 만족스럽지는 않을 수 있습니다. 하지만 무(無)에서 유(有)로 초안을 만들어주는 과정 자체가 큰 도움이 됩니다.

이 상태에서 스스로 문장을 다듬거나, LLM과 대화를 이어가며 계속 수정해나가면 포트폴리오의 ‘한 덩어리(제목, 그림, 문제, 해결, 결과)’는 보통 1~2시간 안에 작성할 수 있습니다.

여기서 한 스펀 더 없으면 효과가 훨씬 좋아집니다. 제대로 된 조직에서는 기능 하나를 추가했을 때 부하 테스트를 진행하는 경우가 많습니다. 이유는 여러 가지가 있지만, 예를 들어 트래픽이 많은 서비스라면 게이트웨이에서 rate limit을 걸어야 하고, 그 기준을 잡기 위해 성능 테스트를 하는 것은 자연스러운 흐름입니다.

이때 테스트 결과로 수치가 나올 텐데, 그런 데이터를 ‘결과’ 파트에 적어주는 건 정말 좋은 방향성입니다. 예를 들어 몇 코어, 몇 GB 메모리 환경에서 어느 정도 TPS로 부하를 줬는지, 병목이 어디였는지, 개선 후 수치가 어떻게 달라졌는지를 간단히 적어주면 설득력이 확 올라갑니다.

마지막으로 당부드리고 싶은 부분은 ‘구현 코드’를 포트폴리오에 넣지 않는 것을 추천드린다는 점입니다. 위 예시처럼 어노테이션 이름이나 핵심 키워드를 짧게 언급하는 정도는 괜찮지만, 코드를 캡처해서 그대로 붙여 넣는 경우가 있습니다. 주변 분들께 피드백을 받으면 “괜찮다”는 말을 듣기도 하지만 그건 환경이 비슷하기 때문입니다. 취업, 이직하려는 회사는 현재 환경의 역량보다 더 높기 때문에 면접관 입장에서는 오히려 좋지 않게 보는 경우가 많습니다. 불필요한 캡처는 넣지 말고, 코드가 아니라 흐름과 근거(그림, 테스트 결과, 수치)로 보여주는 방향을 추천드립니다.

2장 : 포트폴리오 실전 정복

2-4) 소제목 : 문제 해결 경험이 아는데, 포트폴리오에 써도 되나요?

앞서 ‘2-3) 소제목: 포트폴리오 쓰는 방법: 그림, 문제 해결’에서 포트폴리오를 작성하는 방법을 배워봤습니다. 그런데 포트폴리오에는 ‘문제 해결 결과’만 써야 할까요? 다른 내용은 쓸 수 없는 걸까요?

결론부터 말하면, 꼭 그렇지 않습니다. 프로젝트를 기획하고 만드는 과정 자체가 이미 문제 해결이기 때문입니다. 즉, 모든 프로젝트를 “문제 해결” 관점으로 풀어낼 수 있습니다.

예를 들어 Spring WebFlux를 사용해 SSE 프로젝트를 구축했다고 가정해보겠습니다. 이 프로젝트를 시작하고 구축한 것 자체가, 회사 내부에 “해결해야 할 문제”가 있었다는 의미입니다.

그리고 그 문제를 해결하기 위한 방법으로 “Spring WebFlux 기반 SSE”를 선택한 것이죠. 따라서 이 사례도 충분히 ‘아키텍처 그림, 문제, 해결, 결과’의 형태로 정리할 수 있습니다.

여기서 중요한 이야기를 하나 더 하려고 합니다. 1장부터 계속 반복해서 강조한 내용은 결국 ‘문제 해결 역량’입니다. 그래서 “문제 해결이 전부다”라고 느껴질 수도 있습니다.

그런데 취준생분들과 많은 실무자분들이 가장 어려워하는 지점이 있습니다. 프로젝트를 진행해왔는데, “문제 해결이라고 부를 만한 게 없었다”고 느끼는 겁니다. 그러다 보니 스스로를 자책하게 됩니다.

“지금까지 내가 한 건 전부 물경력이었구나.”
“학교나 스터디에서 했던 건 다 잘못됐구나.”

이런 생각 때문에 힘들어하시는 분들도 많습니다. 하지만 자책하실 필요는 없습니다. 해결책이 있습니다. 이럴 때 많은 사람들이 “사이드 프로젝트를 해야 한다”라고 말하곤 하는데, 꼭 그게 정답은 아닙니다. 지금까지 했던 프로젝트가 로컬에서라도 실행만 될 수 있다면, 그걸로 충분합니다.

그 프로젝트를 다시 실행해보고, 개선할 수 있는 포인트를 하나씩 리스트업해보세요. 리스트업의 기준은 간단합니다. “트래픽이 1000TPS 정도 들어오면 어떤 문제가 생길까?”라는 가정을 세우는 겁니다. 즉, 실제 트래픽이 없어도 상상해보는 거예요. “이 정도 부하가 오면 어디가 먼저 터질까?” “병목은 어디에서 생길까?” 이렇게 질문을 던지면서 하나씩 찾아내고, 개선 포인트를 정리해보시면 됩니다. 그리고 그것을 해결하고 이력서에 추가로 써주시면 됩니다.

3장 : 자기소개 실전 정복

3-1) 소제목 : 자기소개서 쓰는 방법

플랫폼을 통해 지원하는 것이 아니라, 직접 회사 홈페이지에 들어가 지원하는 경우 회사에서 요구하는 자기소개 질문을 자주 만나게 됩니다. 이번에는 그 자기소개를 어떻게 작성하면 좋은지에 대해 말씀드리려고 합니다.

저는 이 방법을 많은 분들이 이미 알고 있을 거라고 생각했습니다. 그런데 멘토링을 수백 번 진행하고, 지인들의 이력서와 제 면접관 경험을 통해 지원자들을 보다 보니 의외로 이 방법을 모르는 분들이 많았습니다. 그래서 지금부터 이 방법을 공유드리려고 합니다. 이 글을 읽은 분들은 알고 활용할 수 있는 방식이기 때문에, 실제 지원 과정에서 경쟁력이 될 거라고 확신합니다.

그럼 아래 예시를 보면서 자기소개 작성 방법을 함께 시작해볼까요?

지원 직무에서의 본인이 발휘할 수 있는 강점이나 경험을 구체적인 사례를 들어 서술해 주시고, 이를 바탕으로 생각등대컴퍼니에서 이루고 싶은 목표가 무엇인지 서술해 주세요. (1,000자 이내)

[Kafka, Redis, Websocket 이용 및 차세대 금융 시스템 구축 => 하루 1200만건 실시간으로 빠른 서비스 제공]

차세대 생각 시스템을 구축하는 프로젝트에서 대용량 데이터를 안정적으로 처리하고 실시간으로 제공하는 백엔드 아키텍처 설계에 기여를 하였습니다. 기존에는 Polling 기반의 데이터 요청 구조로 인해 네트워크 트래픽과 서버 부하가 과도하게 발생했으나 이를 WebSocket-STOMP 기반의 실시간 데이터 전송 구조로 전환하여 지연 시간을 줄이고 서버 비용을 절감하는 성과를 이루었습니다. 또한, Kafka Streams와 Redis Pub/Sub을 활용하여 실시간 데이터 처리 속도를 극대화하고 WebSocket의 Snapshot 기능을 통해 트래픽을 최적화하며 전반적인 시스템 성능을 개선했습니다.

특히, 하루 1200만 건 이상이 실시간으로 발생하는 생각 데이터를 Kafka와 Redis 기반 파이프라인으로 처리하고 과거 데이터를 주기적으로 적재하는 구조를 설계하여 시스템 부하를 최소화하면서도 안정적으로 데이터를 제공할 수 있도록 했습니다.

[Polling 구조가 아닌 실시간성을 제공하고자 Webflux 를 이용한 SSE 도입 => 생각등대 컴퍼니 서비스의 리뷰, 댓글 상태 정보 개선에 기여]

생각 시스템 영역에서도 기존의 Polling 기반 처리 방식을 WebFlux 기반의 Reactive 비동기 스트림 구조로 개선하여 리소스 낭비를 줄이고 실시간 처리 성능을 향상시켰으며 이를 통해 사용자 서비스 경험의 체감 속도 또한 개선되었습니다. 이 외에도, 다양한 외부 시스템과의 연계를 고려한 메시지 큐 아키텍처 설계 경험과, 실시간 알림 시스템 구축 등을 통해 확장성과 안정성을 모두 고려한 백엔드 시스템을 구현한 경험이 있습니다.

생각등대컴퍼니의 다양한 서비스를 이용해보니 리뷰, 댓글 상태에 대한 부분을 실시간으로 업데이트 되지 않는 부분을 확인하였습니다. 이와 같은 실시간 데이터 처리 및 안정적인 시스템 아키텍처 역량이 충분히 기여할 수 있을 것이라 확신합니다.

[그림] 자기소개서 예시

이해를 돕기 위해 비교적 쉬운 예시를 가져왔다는 점 참고 부탁드립니다. 위의 그림을 보면서 두 가지를 말씀드리려고 합니다.

첫 번째 방향성은 ‘500자씩 문단 나누기’입니다. 앞서 말씀드렸듯, 이력서를 보는 실무진들은 매우 바쁜 일상을 보내고 있습니다. 인력이 부족해서 채용 공고를 올린 상황에서, 이력서를 검토하고 과제를 확인해야 하며, 업무 시간 중에 면접까지 진행합니다. 게다가 불합격을 결정한 경우에는 그에 대한 타당한 이유를 인사팀에 정리해 전달해야 하는 경우도 있습니다.

즉, 이런 환경에서 자기소개 항목은 실무진 입장에서 가장 부담이 큰 파트가 되기 쉽습니다. 하루에 적어도 10개 이상의 이력서를 보는 경우가 많기 때문에, 모든 자기소개를 처음부터 끝까지 정독하기는 어렵습니다. 현실적으로는 ‘스쳐가듯 훑어본다’고 생각하시면 됩니다.

그렇다면 이런 상황에서 지원자는 자기소개 항목에서 어떻게 어필해야 할까요? 다른 지원자보다 조금이라도 더 높은 점수를 얻기 위해서는, 무조건 전략적으로 접근해야 합니다.

500자씩 문단을 나눠주세요. 위 예시처럼 1,000자 분량의 질문이라면 500자씩 두 문단으로 나누면 됩니다. 그리고 각 문단은 서로 다른 주제를 담아 어필해야 합니다. 많은 분들이 1,000자라고 하면 처음부터 끝까지 한 문단으로 길게 이어서 작성하는데, 면접관 입장에서는 읽는 부담이 커지고, 자연스럽게 읽고 싶지 않은 자기소개서가 되기 쉽습니다. 반대로 500자씩 끊어서 작성하면 부담이 확 줄어듭니다.

그래서 무조건 500자씩 문단을 나눠서 작성하는 것을 추천드립니다.

두 번째 방향성은 ‘소제목’입니다. 어떻게 보면 첫 번째 방향성의 연장선입니다. 500자씩 문단을 나눴다면, 그중 약 50자는 소제목으로 사용하면 됩니다. 즉, 50자(소제목) + 450자(내용) 구조로 작성하는 방식입니다.

소제목은 크게 두 가지 형태로 쓸 수 있습니다.

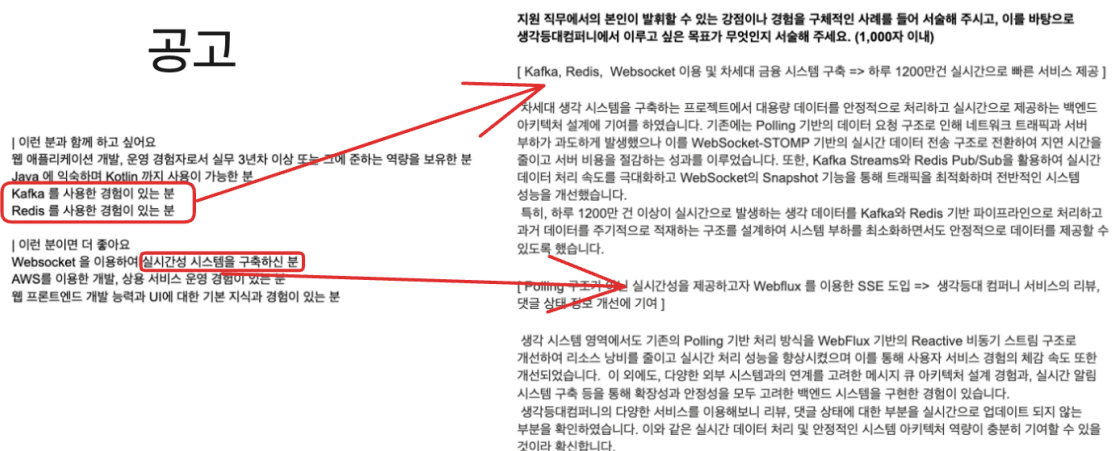
- “어떤 프로젝트/경험을 통해 팀에 어떤 도움을 줄 수 있다” 형태로 쓰는 방법입니다.
- “어떤 경험을 통해 내가 어떻게 성장했는가” 형태로 쓰는 방법입니다.

이렇게 작성하면 면접관 입장에서는 450자 내용을 전부 읽지 않더라도, 50자 소제목만 보고도 글의 핵심을 빠르게 파악할 수 있습니다.

정리하자면, 이제 위의 자기소개 예시를 보면 전체 구조를 어떻게 잡아야 하는지 감이 오실 겁니다. 1,000자를 전부 읽지 않더라도 핵심만 추린 소제목 두 줄 즉, 100자만으로도 지원자의 자기소개를 쉽게 파악할 수 있게 만드는 것이 굉장히 좋은 전략입니다.

3장 : 자기소개 실전 정복

3-2) 소제목 : 합격률을 높이는 필살기



[그림] Job Description 과 자기소개서

바로 전 챕터 ‘3-1) 소제목: 자기소개서 쓰는 방법’에서 자기소개서를 작성하는 방법을 알려드렸습니다. 이제는 자기소개서 합격률을 높이는 저만의 필살기를 알려드리려고 합니다.

결국 자기소개서의 모든 질문은 ‘지원 자격’, ‘우대사항’, 즉 Job Description에 맞게 작성하는 것이 가장 중요합니다. 이제 위의 그림을 보면서 예시로 설명드리겠습니다.

왼쪽을 보면 해당 회사의 Job Description이 있습니다. 내용을 보면 'Kafka를 사용한 경험이 있는 분', 'Redis를 사용한 경험이 있는 분'이라고 되어 있습니다. 그리고 오른쪽의 자기소개서 질문을 보면 "지원 직무에서 본인이 발휘할 수 있는 강점이나 경험을 구체적인 사례로 작성하라"는 내용입니다.

이 질문을 보고, 이전 챕터에서 말씀드린 방식대로 500자씩 덩어리로 나누고, 각 덩어리마다 소제목을 50자 정도로 붙인다고 가정해 보겠습니다. 그러면 1,000자 기준으로 2개의 덩어리가 나오겠죠. 여기서 중요한 건, 각 덩어리를 JD의 요구사항에 맞게 '매핑'하는 것입니다.

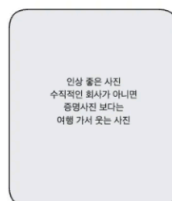
예를 들어 첫 번째 500자 덩어리는 'Kafka를 사용한 경험'과 매핑되는지 내 프로젝트 경험을 먼저 확인한 뒤, 그에 맞는 소제목과 내용을 작성합니다. 두 번째 500자 덩어리도 동일하게 'Redis를 사용한 경험'이 내 프로젝트에서 매핑되는지 확인한 다음, 같은 방식으로 소제목과 내용을 작성하면 됩니다.

자, 자기소개서의 모든 내용을 이런 식으로 Job Description에 맞춰 작성하는 겁니다. 그러면 자기소개서가 완성되는 순간, 그 회사와 그 팀이 원하는 '딱 맞는 지원자'가 됩니다.

4장 : 다른 지원자와 차별화되는 '나만의 무기' 만들기

4-1) 소제목 : 뻔한 블로그를 나만의 무기로 만드는 방법

생각등대 이력서 (멘토링 교육용)



Introduction

지원 동기

Java 기반의 웹 백엔드 기술을 중심으로 실무형 프로젝트와 교육을 수료한 신입 개발자입니다. JSP, Spring Framework, jQuery를 활용한 웹 기능 개발 경험이 있으며, 실습 과정에서 Git과 Maven을 이용한 협업과 빌드 경험도 보유하고 있습니다. 그룹웨어처럼 다양한 사용자 요구에 맞춘 기능 커스터마이징과 유지보수가 중요한 서비스에서 실용적이고 유연한 개발자로 성장하고 싶어 지원하게 되었습니다.

책임감 있는 개발자

팀 프로젝트를 진행하며 일정 관리에 대한 책임감을 가장 중요한 가치로 생각하게 된 개발자입니다. Redis 기반의 캐시 처리와 MySQL 데이터 모델링, Query 튜닝에 강점을 가지고 있으며, 실습 중심의 프로젝트에서 백엔드 구조를 설계하고 구현하는 경험을 쌓아왔습니다. 팀 내에서는 주어진 일정을 끝까지 책임지고 마무리하며 기획자, 디자이너와의 원활한 커뮤니케이션을 통해 팀원들로부터 신뢰를 받아왔습니다.

Blog Link : 레디스, 나는 끝까지 파고들었다. 레디스, 더 깊이 들어갈 데가 없다.

Personal Information

NAME : 생각등대

Company (or Education)

- B_생각등대 컴퍼니 (취업 개발팀) : 2025.06 ~ 현재
- A_생각등대 컴퍼니 (취업 개발팀) : 2020.12 ~ 2022.10
- 생각등대 부트캠프 (Java 기초 및 실무 교육) : 2020.02 ~ 2020.05

GITHUB : 생각등대 GitHub URL

BLOG : 생각등대 Blog URL

PORTFOLIO : <https://think-lighthouse.com/> (본인이 선택한 포트폴리오 URL)

[그림] 이력서 최상단 - Blog Link

나만의 무기는 무조건 있어야 합니다. 수요보다 공급이 많아지는 개발자 시장에서는 ‘유니크함’이 필요합니다. 그 유니크함을 만드는 방법은 여러 가지가 있지만, 지금 제가 알려드리는 방식은 **나만의 강점을 강하게 어필할 수 있는 강력한 무기**입니다. 또 반대로 나의 약점을 강점으로 만들 수 있는 무기입니다.

특히 회사에 다니면서도 이직할 때 내세울 만한 프로젝트 경험이 부족하다고 느끼는 경우, 또는 SI에서 서비스 기업으로 이직을 목표로 하는 경우에 가장 효과적으로 쓸 수 있는 무기라고 확신합니다. 꼭 활용해 보셨으면 좋겠습니다.

위의 [그림] 예시를 한번 보겠습니다. 예시를 보면 Introduction 아래, Personal Information 위에 뜬금없이 ‘Blog Link’가 있습니다. 이건 의도된 배치입니다. 이력서를 보는 면접관 입장에서는 흐름과 다른 위치에 한 줄이 툭 튀어나와 있기 때문에, 시선이 갈 수밖에 없습니다. 즉, “무조건 한 번 눌러보게” 유도한 자리인 겁니다.

더 나아가 그 한 줄 문구가 “Redis, 나는 끝까지 파고들었다. Redis, 더 깊이 들어갈 데가 없다.”라면 어떨까요? 면접관 입장에서는 눈길이 갈 수밖에 없고, 클릭하지 않을 이유가 없습니다.

다만, 여기서 중요한 포인트가 있습니다. 이렇게 링크 클릭을 유도해 기대감을 만들어 놓았다면, 이제 그 기대를 ‘한 방’으로 충족시켜야 합니다. 기대를 크게 올려놓고 내용이 약하면 오히려 반감만 살 수 있습니다.

자, 이제 어떻게 써야하는지 본격적으로 말씀드리겠습니다. 저는 서비스 회사를 다녔습니다. 서비스 회사를 다니면 테크 블로그는 무조건 있습니다. 테크 블로그 게시물을 보면 굉장히 잘 쓰지 않았나요? 잘 쓴 이유가 있습니다.

- 회사의 이름이 걸려있습니다
- 테크 블로그를 쓰면 보편적으로 회사에서 돈을 줍니다
- 글 자체가 회사 소속이 되는 것이라 최대한 잘못된 정보를 주지 않기 위해 씁니다

이런 이유로 글 하나를 쓰는 데 3주에서 4주 정도 걸리는 것 같습니다. 이렇게 시간을 들여 쓴 글은 자연스럽게 퀄리티가 높아질 수밖에 없습니다. 실제로 이런 글을 작성한 실무진들은 해당 글을 활용해 이직에 성공하기도 합니다. 그만큼 경력자에게도 도움이 되는 강력한 무기입니다.

그리고 이런 글은 한 개면 충분합니다. [그림]에 나온 예시처럼 제목도 눈길이 가도록 톱 던져 주세요. 면접관이 이 글을 클릭하는 순간, “이 지원자는 역량이 높다”라고 판단할 가능성이 커지고, 합격 확률도 크게 올라갑니다.

그렇다면 어떤 글을 써야 할까요? 저는 Redis, MySQL처럼 많은 회사에서 공통적으로 사용하는 기술을 주제로 작성하는 것을 추천드립니다. 그래야 이 무기를 다양한 회사에 폭넓게 적용할 수 있습니다.

여기서 정말 추천드리고 싶은 방향성이 있습니다. 지금 내가 잘하고 있는 강점을 쓰는 것도 충분히 강력한 무기가 될 수 있습니다. 하지만 저는 오히려 반대로, 내 상황을 보고 면접관이 “아... 이 부분은 좀 아쉽다”라고 느낄 만한 지점을 그 무기로 커버하는 방향을 더 추천드립니다.

즉, 단점을 숨기거나 피하는 게 아니라, 그 약점 자체를 깊게 파고든 글로 증명해 보이는 겁니다. 이렇게 하면 단점이 자연스럽게 보완되면서, 전체적으로 훨씬 완성도 높은 지원자로 보일 수 있습니다. 저는 이 무기를 ‘단점을 커버하는 용도’로 활용하는 전략이 특히 효과적이라고 생각합니다.

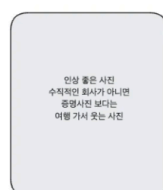
예를 들어, SI에서 서비스 회사로 이직을 준비하는 분들 중에는 MySQL보다 Oracle을 주로 사용해 온 경우가 생각보다 많습니다. 이런 상황이라면 MySQL을 주제로 깊이 있는 글을 하나 써두는 것만으로도 큰 도움이 될 수 있습니다.

4장 : 다른 지원자와 차별화되는 ‘나만의 무기’ 만들기

4-2) 소재목 : 이력서의 소개 섹션을 나만의 무기로 만드는 방법

이력서 최상단에 들어가는 ‘자기 소개’ 카테고리에는 저에게 정말 ‘효자 같은’ 카테고리였습니다. 저는 이 영역을 중심으로 전략을 만들었고, 그 전략 덕분에 많은 기업에 합격할 수 있었습니다. 그래서 오늘은 그 전략을 여러분께 공유하려고 합니다. 특히 취준생, 주니어 분들에게는 정말 큰 무기가 될 수 있습니다. 처음에는 어렵게 느껴질 수 있지만, 꼭 한 번 적용해 보셨으면 좋겠습니다.

생각등대 이력서 (멘토링 교육용)



Introduction

지원 동기

Java 기반의 웹 백엔드 기술을 중심으로 실무형 프로젝트와 교육을 수료한 신입 개발자입니다. JSP, Spring Framework, jQuery를 활용한 웹 기능 개발 경험이 있으며, 실습 과정에서 Git과 Maven을 이용한 협업과 빌드 경험도 보유하고 있습니다. 그룹웨어처럼 다양한 사용자 요구에 맞춘 기능 커스터마이징과 유지보수가 중요한 서비스에서 실용적이고 유연한 개발자로 성장하고 싶어 지원하게 되었습니다.

책임감 있는 개발자

팀 프로젝트를 진행하며 일정 관리에 대한 책임감을 가장 중요한 가치로 생각하게 된 개발자입니다. Redis 기반의 캐시 처리와 MySQL 데이터 모델링, Query 튜닝에 강점을 가지고 있으며, 실습 중심의 프로젝트에서 백엔드 구조를 설계하고 구현하는 경험을 쌓아왔습니다. 팀 내에서는 주어진 일정을 끝까지 책임지고 마무리하며 기획자, 디자이너와의 원활한 커뮤니케이션을 통해 팀원들로부터 신뢰를 받아왔습니다.

Blog Link : 레디스, 나는 끝까지 파고들었다. 레디스, 더 깊이 들어갈 데가 없다.

Personal Information

NAME : 생각등대

Company (or Education)

- B_생각등대 컴퍼니 (취업 개발팀) : 2025.06 ~ 현재
- A_생각등대 컴퍼니 (취업 개발팀) : 2020.12 ~ 2022.10
- 생각등대 부트캠프 (Java 기초 및 실무 교육) : 2020.02 ~ 2020.05

GITHUB : 생각등대 GitHub URL

BLOG : 생각등대 Blog URL

PORTFOLIO : <https://think-lighthouse.com/> (본인이 선택한 포트폴리오 URL)

[그림] 이력서 최상단 - 자기 소개

위의 [그림]을 같이 볼까요? 지원 동기 아래에 ‘자기소개’ 카테고리가 있습니다. 저는 이 부분을 무조건 ‘첫 질문’이 나오도록 설계합니다. 지금 보여 드린 예시는 다소 아쉬울 수 있지만, 실제 이력서에는 훨씬 더 강하게 쓰는 편입니다.

예를 들어, “데이터 파이프라인을 설계해 수천만 건의 데이터를 실시간으로 처리할 수 있게 했고, 그 과정에서 어떤 성과를 냈다”처럼 한 줄만 읽어도 궁금증이 생기게끔 작성합니다. 그러면 면접관이 자연스럽게 “생각등대님, 여기 써 있는 내용은 어떤 프로젝트인가요?”라고 첫 질문을 던지게 됩니다. 저는 그 질문이 나오도록 의도적으로 유도하는 편입니다.

의도한 대로 첫 질문이 나온다면, 준비한 대로 이렇게 답변합니다.

“아, 그건 Kafka를 이용해서요. 혹시 뒤에 화이트보드가 있다면, 아키텍처를 그리면서 설명드려도 될까요?” 그리고 곧바로 화이트보드에 아키텍처를 그리면서 설명을 시작합니다.

면접관 입장에서는 어떨까요? 제가 면접관 경험을 해봤을 때, 아키텍처를 직접 그리겠다고 말하는 지원자는 정말 한두 분밖에 없었습니다. 즉, 그림을 잘 그리든 못 그리든 그 자체로 엄청난 임팩트가 됩니다. 또, 화이트보드가 없을 수 있습니다. 이럴 때, 아이패드도 같이 들어가는 치밀함도 가지고 면접을 진행합니다.

자, 어떠신가요? 아키텍처를 그리면서 설명한다는 건 분명 어렵고 부담될 수 있습니다. 하지만 그만큼 강력한 무기입니다. 제가 면접관으로 참여했을 때도, 아키텍처를 직접 그리며 설명하던 한두 분은 정말 강렬해서 아직도 기억에 남아 있습니다.

처음엔 부담스럽더라도, 가능하다면 꼭 이 무기를 활용해 보셨으면 좋겠습니다.

5장 : 지원 전략과 AI 시대 생존법

5-1) 소재목 : 지원하는 방법. 아직도 플랫폼에서 지원하시나요?

취업, 이직할 때 보편적으로 가장 먼저 하는 일은 무엇일까요? 아마 대부분은 여러 채용 플랫폼을 열어 공고부터 살펴보실 겁니다. 사이트도 정말 많고, 빠른 지원 같은 편리한 기능도 다양하죠.

그런데 지원하기 전에, 면접관의 입장에서는 이 지원서가 어떻게 보일지 한 번쯤 생각해보셨나요? 그 시선을 먼저 이해해야 우리가 어떻게 지원해야 하는지도 보입니다. 결국, 남들과는 달라야 합니다. 그럼 시작해보겠습니다.

면접관 경험을 하게 되면 여러 채용 플랫폼을 통해 들어오는 이력서를 받게 됩니다. 또 다른 방식으로는, 회사 홈페이지를 통해 직접 지원한 지원자의 이력서도 보게 되죠. 이 두 가지는 정말 다릅니다. 왜 다를까요?

가장 쉽게 설명해보겠습니다. 겨울을 떠올려보세요. 아주 추운 날, 우리는 보통 무엇을 입나요? 검정색 롱패딩을 입고 다니죠. 밖을 나가보면 전부 다 검정색 롱패딩입니다. 횡단보도에 서있는 대다수가 검정색 롱패딩입니다. 딱 그 느낌입니다. 채용 플랫폼을 통해 지원한 이력서들은 첫인상이 마치 검정 롱패딩을 보고 있는 것처럼 비슷하게 느껴지는 경우가 많습니다. 그래서 첫인상에서 강한 메리트를 주기 어렵습니다.

우리는 모두 알고 있습니다. 이력서는 정말 짧으면 1분 안에 전체 윤곽이 파악되고, 길어도 5분이면 합격, 불합격의 판단이 나옵니다. 즉, 첫인상이 중요합니다. 그런데 첫 이미지가 검정 롱패딩처럼 다른 지원자와 차별점이 없다면, 과연 이게 올바른 지원 방법일까요?

다른 누군가는 회사 홈페이지에 직접 들어가 지원하면서 확실한 차별점을 만들기도 합니다. 그런데 왜 이런 차이가 생길까요?

첫번째, 채용 플랫폼의 고정된 이력서 포맷이 아니라 ‘나만의 이력서’를 만들어 제출하기 때문에 더 눈에 띕니다. 대부분의 채용 플랫폼은 플랫폼 자체의 이력서 양식이 정해져 있어, 지원자들이 거의 비슷한 형태로 제출하게 됩니다. 반면 회사 홈페이지로 직접 지원하면 워드, 구글 독스, 노션 등 원하는 형식으로 이력서를 구성해 제출할 수 있습니다. 자연스럽게 ‘눈길이 가는 이력서’가 됩니다.

두번째, 회사마다 요구하는 내용이 다르기 때문입니다. 예를 들어 어떤 회사는 채용 플랫폼에서는 클릭 몇 번으로 바로 지원이 가능하지만, 회사 홈페이지로 들어가면 1,000자 자기소개나 추가 질문을 반드시 작성하도록 요구하기도 합니다. 그렇다면 면접관 입장에서는 당연히 그 1,000자를 성실하게 작성한 지원자에게 더 관심이 가게 됩니다.

이해되셨죠?

그럼 왜 채용 플랫폼에서 공고를 홍보하는 걸까요? 무조건적인 정답은 아니지만, 보편적으로는 인사팀과 실무진의 관점이 다르기 때문입니다.

인사팀 입장에서는 ‘지원자가 많이 들어오는 것’ 자체가 성과가 될 수 있습니다. 그래서 여기저기 공고를 홍보하고, 심지어 생각등대컴퍼니 오픈카톡에서도 채용 담당자분들이 홍보를 하기도 하죠. 즉, 인사팀은 ‘많은 지원’을 목표로 움직이고, 실제로 이력서를 검토하는 실무진은 목적이 다르기 때문에 이런 현상이 생기는 겁니다.

그래서 저는 가능하면 회사 홈페이지로 직접 지원하는 것을 추천드립니다. 다만 “그 회사 홈페이지에 지원 메뉴가 없으면 어떻게 하죠?”라는 질문이 나올 수 있겠죠. 그럴 때는 어쩔 수 없이 채용 플랫폼을 이용하되, 플랫폼에서 요구하는 빈칸들은 이력서 내용과 일부 겹치더라도 성실하게 채워주세요. 그리고 별도로 본인의 이력서를 링크로 첨부하거나 PDF 파일로 함께 제출하는 것을 추천드립니다.

5장 : 지원 전략과 AI 시대 생존법

5-2) 소재목 : AI 로 인해 어떻게 살아남아야 할까?

마지막 챕터에서는 AI에 대해 어떻게 전략적으로 방향을 잡아야 하는지, 제 개인적인 의견을 말씀드리려고 합니다. 정답은 아닙니다. 다만 저는 이렇게 준비하고 있습니다.

많은 취업 준비생분들과 실무자분들이 AI 때문에 앞으로 어떻게 이 흐름을 헤쳐 나가야 할지 깊이 고민하고 있다는 것을 알고 있습니다. 우선, 고민을 시작했다는 것 자체가 이미 다른

누구보다 앞서 나가고 있다는 뜻입니다. 이제는 더 앞서 나갈 때입니다. 결국 실천해야 하니까요. 그렇다면 어떻게 헤쳐 나가야 할까요?

첫 번째 방향성은 ‘회사에 대한 의존성 끊기’입니다. 제가 대표라면 AI를 통해 자동화를 가장 먼저 시작할 것 같습니다. 그리고 그 과정에서 인력 감소도 자연스럽게 따라올 가능성이 높겠죠. 당연한 흐름 아닐까요? 이 글을 읽는 독자분들이나 개발자분들이 대표라고 가정해도, 비슷한 판단을 하실 거라고 생각합니다.

물론 어떤 회사는 “우리는 AI로 절감한 시간만큼 더 중요한 일에 투자할 테니 걱정하지 마라”라고 말할 수도 있습니다. 하지만 그 말을 그대로 믿는 것은 순진한 선택일 수 있다고 봅니다. 결국 중요한 건, 어떤 상황에서도 흔들리지 않도록 ‘회사에 대한 의존성’을 끊어내기 위해 고민하고 준비하는 것. 저는 그게 첫 번째라고 생각합니다.

그래서 회사가 아닌 나만의 힘으로 밖에서 한달에 1만원이라도 한번 벌어보는 것 부터 시작하셨으면 좋겠습니다. 그로 부터 오는 경험이 여러 생각이 들게할것이라고 단언할 수 있습니다.

마지막 방향성은 ‘AI 툴을 잘 쓰는 법’이 아니라, 오히려 ‘기본기를 더 탄탄히 다지는 것’입니다. 요즘 유튜브나 여러 SNS를 보면, 다양한 AI 툴을 빠르게 익히고 익숙해지라는 영상과 강의가 정말 많이 나옵니다. 물론 직접 사용해보고 익숙해지는 과정은 좋은 경험이고, 분명 남는 것도 있다고 생각합니다. 하지만 그게 과연 가장 좋은 방향이냐고 묻는다면, 저는 아니라고 봅니다.

지금의 AI는 아직 변화 속도가 너무 빠른 시기입니다. 하루아침에 새로운 툴이 쏟아져 나오고, 그때마다 또 익혀야 합니다. 그런데 뭔가 이상하지 않나요? 나오는 툴마다 공부하고, 배우고, 강의 듣고... 이게 맞을까요? 저는 이 방향은 비효율적이라고 생각합니다.

오히려 시간이 조금 지나서 ‘끝까지 살아남는 AI’가 어느 정도 정리되었을 때, 그때 필요한 강의를 듣고 체계적으로 배우는 편이 더 효율적일 수 있습니다. 그렇다고 해서 지금 아무것도 하지 말라는 뜻은 아닙니다.

지금 해야 할 일은 따로 있습니다. 다양한 툴이 존재한다는 건, 그 안에 ‘공통된 코어 개념’이 있다는 뜻입니다. 그 공통된 핵심을 공부하셔야 합니다. 쉽게 말해, 바뀌지 않는 부분을 먼저 배우는 것이 중요합니다. 공부 방향성을 다시 잡아주세요.

맺음말

멘토링을 하면서 생각보다 더 많은 분들께 도움을 드릴 수 있다는 걸 알게 됐습니다. 하지만 많은 분들이 원하시더라도 시간적 한계 때문에 모든 분들께 충분한 정보를 제공하기는 어렵다는 점도 깨달았습니다. 그래서 그 해결책으로 전자책을 선택했습니다.

멘토링을 신청하는 일은 누군가에게 큰 용기가 필요한 선택일 수 있습니다. 이력서와 포트폴리오를 보여주는 것 자체가 부끄럽게 느껴질 수도 있고, 대면이나 음성으로 첨삭을 받는 과정이 부담스러울 수도 있습니다. 저 역시 그런 마음을 잘 압니다.

그래서 전자책은 실시간 멘토링처럼 1:1로 진행하지 않더라도, 더 많은 분들께 도움을 드릴 수 있는 방식이라고 판단했습니다. 그 결과 전자책 제작을 빠르게 실행에 옮겼습니다.

단언컨대, 이 전자책은 제가 멘토링에서 가장 중요하다고 생각하는 핵심만 모아 정리한 내용입니다. 실시간 멘토링과의 차이는 개인 맞춤형 피드백이 아니라는 점뿐이며, 전달하는 핵심은 본질적으로 동일합니다.

멘토링을 하며 가장 많이 마주한 고민은 ‘반복되는 서류 탈락’이었습니다. 50개, 100개의 지원서를 작성해도 합격률이 10%도 되지 않아 좌절감과 자신감 하락을 겪는 분들이 정말 많았습니다. 이 문제는 생각보다 삶 전체에 큰 영향을 주기도 합니다. 그래서 저는 이 고민을 가볍게 보지 않습니다.

이 전자책 역시 그런 마음으로, 취업 준비생과 이직 준비생분들의 서류 합격률을 최대한 높이는 데 도움이 되도록 필요한 정보를 최대한 담았습니다.

© 2026 생각등대. All rights reserved.

이 자료는 대한민국 저작권법의 보호를 받습니다. 작성된 모든 내용의 권리는 작성자에게 있으며, 작성자의 동의 없는 사용이 금지됩니다.

본 자료의 일부 혹은 전체 내용을 무단으로 복제/배포하거나 2차적 저작물로 재편집하는 경우, 5년 이하의 징역 또는 5천만원 이하의 벌금과 민사상 손해배상을 청구합니다.

※ 저작권법 제 30조(사적이용을 위한 복제)

공표된 저작물을 영리를 목적으로 하지 아니하고, 개인적으로 이용하거나 가정 및 이에 준하는 한정된 범위 안에서 이용하는 경우에는 그 이용자는 이를 복제할 수 있다. 다만, 공중의 사용에 제공하기 위하여 설치된 복사기기에 의한 복제는 그러지 아니하다.

※ 저작권법 제 136조(벌칙)의 ① 다음 각 호의 어느 하나에 해당하는 자는 5년 이하의 징역 또는 5천만원 이하의 벌금에 처하거나 이를 병과할 수 있다.

지적재산권 및 이 법에 따라 보호되는 재산적 권리(제93조에 따른 권리는 제외한다)를 복제, 공연, 공중송신, 전시, 배포, 대여, 2차적 저작물 작성의 방법으로 침해한 자

※ 민법 제750조(불법행위의 내용)

고의 또는 과실로 인한 위법행위로 타인에게 손해를 가한 자는 그 손해를 배상할 책임이 있다.